Project Interim Report

On

Hospital Appointment Booking System

(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming with Java

SESSION: 2025-26

Submitted By

Student name:

Umar Hayat Khan

ERP:- RU-25-11791

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with IBM

Under the Guidance of

MR. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI, CG

(April 2026)

Abstract

TheHospitalAppointment Booking System is a Java-based application developed using Object-Oriented Programming (OOP) principles. It addresses the real-world challenge of managing patient appointments efficiently in a hospital environment. Manual scheduling of appointments often leads to time conflicts, double bookings, and poor resource utilization. This system eliminates such issues by automating the booking and cancellation process.

The system maintains a structured record of doctors, patients, and appointments. Using an Appointment Manager class, it checks time slot availability before confirming any booking, thus preventing scheduling conflicts. If a requested time slot is already occupied by a doctor, the system immediately notifies the user with an appropriate error message. Appointments can also be cancelled at any time, freeing the slot for other patients.

The project uses four interacting classes — Doctor, Patient, Appointment, and Appointment Manager — each responsible for a specific part of the system. Data is stored and managed using arrays of objects. This project demonstrates key OOP concepts including class interaction, object composition, encapsulation, and array-based data management, providing students with practical experience in designing modular, real-world Java applications.

Introduction

Inmodern hospitals,scheduling patient appointments manually is a time-consuming and error-prone task. Doctors have limited workinghours and specific time slots for consultations. Managing these slots across multiple doctors and hundreds of patients requires a reliable, organized, and conflict-free system.

The Hospital Appointment Booking System is designed to solve this problem using Core Java and Object-Oriented Programming concepts. The system provides a digital solution to book, cancel, and view appointments while ensuring no two patients are assigned to the same doctor at the same time. The application uses interacting Java classes to model real-world entities such as doctors, patients, and appointments.

This project was developed as part of the Object Oriented Programming with Java course (RU-100-01-00012) at Rungta International Skills University. It is a console-based application that simulates the core functionality of a hospital scheduling system, reinforcing the following key Java and OOP skills:

Designing and implementing multiple interacting classes

- Using object composition to link related entities
- Storing and retrieving data using arrays of objects
- Implementing real-world business logic such as availability checking
- Writing modular, readable, and well-commented Java code

The system closely mirrors real hospital workflows — from checking whether a doctor is free at a given time, to confirming a booking or processing a cancellation. This makes it an ideal project for understanding how OOP principles apply to practical software development.

Objectives

The primary goal of this project is to build a functional Hospital Appointment Booking System that demonstrates real-world application of Object-Oriented Programming in Java. The specific objectives are listed below:

Maintain structured records of doctors, patients, and appointments using object arrays

Allow patients to book time slots with available doctors through a systematic process

Prevent double-booking by checking slot availability before confirming any appointment

Enable cancellation of existing appointments using patient ID and time slot

Display all scheduled appointments with doctor name, patient name, time slot, and current status

Apply core OOP principles: encapsulation, object composition, and class interaction

Manage data efficiently using arrays of Doctor, Patient, and Appointment objects

Produce clean, modular, and well-commented Java source code

Simulate a real hospital scheduling workflow in a console-based Java application

Scope

Current Scope

Thissystemisdesigned for use in small to medium-sized clinics and hospital outpatient departments. It operates as a console-basedJava application and manages appointment scheduling forafixed set of doctors and patients pre-loaded in the system.

- Managesafixed array of Doctor and Patient objects initializedwith sample data
- Supportsbooking of appointments with slot availability validation
- Supportscancellation of appointments by patient ID and time slot
- Displaysallappointments with their current status (Booked/Cancelled)
- Operatesentirely through console-based input and output

Future Scope

The currentimplementation provides a strong foundation that can be extended in the following ways:

- Integration with a relational database (MySQL / PostgreSQL) for persistent data storage
- Development of a graphical user interface (GUI) using Java Swing or JavaFX
- Conversion into a web application using Java Spring Boot and REST APIs

- Addition of role-based login for doctors, patients, and administrators
- Email and SMS notifications for appointment confirmation and reminders
- Dynamic data management using ArrayList instead of fixed-size arrays.

System Requirements

Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Hard Disk	500 MB free disk space
Display	1024 x 768 resolution or higher

Software Requirements

Software	Details
Operating System	Windows 10 / macOS/Linux (Ubuntu)
Java Development Kit	JDK 8 or above
IDE / Editor	VS Code / Eclipse / IntelliJ IDEA / Edit Plus
Compiler	javac (bundled with JDK)
Terminal / Console	Command Prompt / PowerShell / Terminal

Methodology

Thesystem follows a structured step-by-step procedural flow to manage hospital appointments. The complete working methodology is described below:

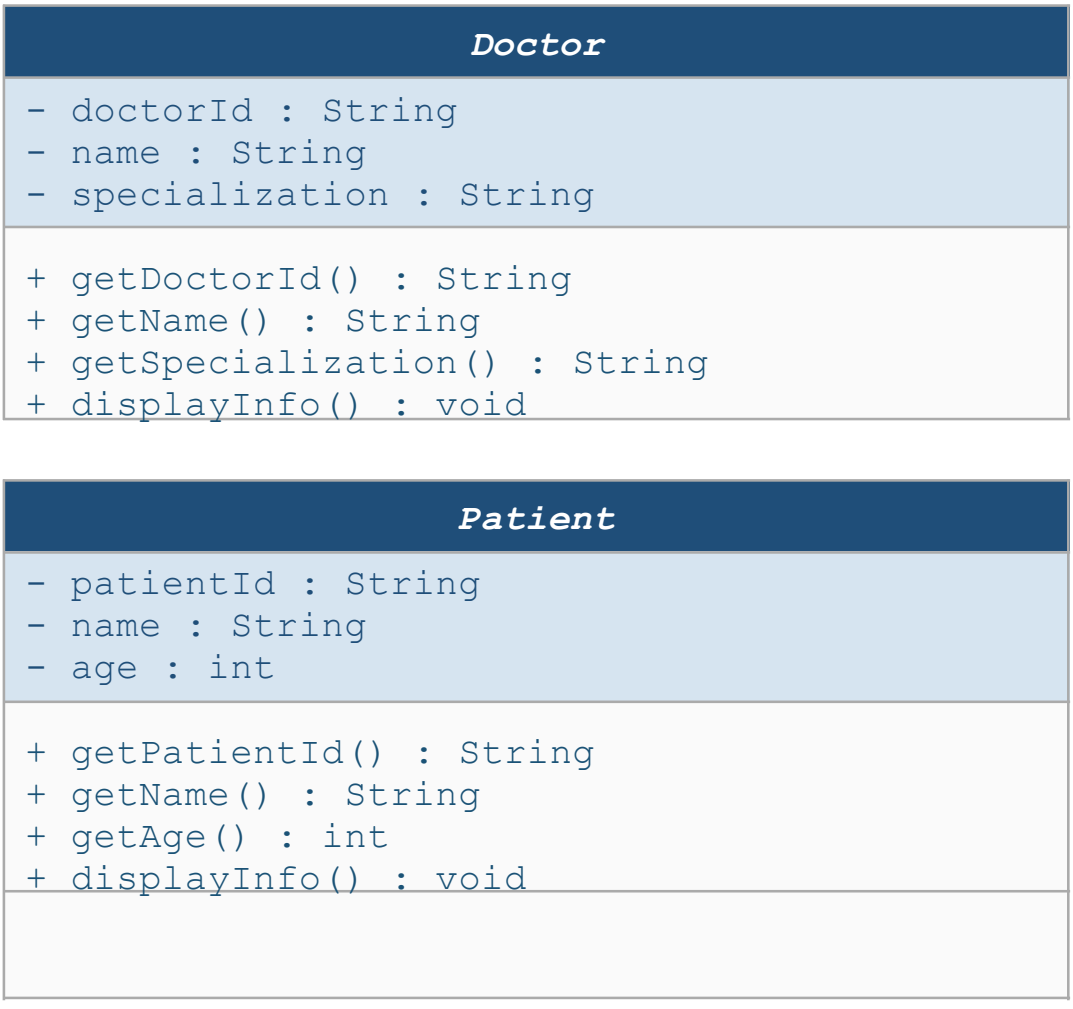
1. Initialization: The system pre-loads sample data consisting of Doctor objects and Patient objects stored in their respective arrays. Sample time slots are also defined for demonstration.
2. Slot Availability Check: Before any booking is made, the AppointmentManager iterates through the existing appointments array to verify that the selected doctor does not already have a confirmed booking at the requested time slot.
3. Booking: If the slot is available, a new Appointment object is created by linking the selected Doctor and Patient objects. It is assigned the requested time slot and stored in the appointments array with status set to "Booked".
4. Conflict Handling: If the time slot is already occupied for the selected doctor, the system prints an error message — "Error: Slot already booked" — and no new appointment is created, preventing any scheduling conflict.
5. Cancellation: To cancel an appointment, the system searches the appointments array using the Patient ID combined with the time slot. Once found, the appointment's status is updated to "Cancelled".
6. Display: The displayAllAppointments() method iterates through the entire appointments array and prints each record — showing Doctor Name, Patient Name, Time

Slot, and current Status — in a formatted layout on the console.

This methodology ensures a logical and conflict-free appointment management flow from booking to cancellation, reflecting real hospital scheduling practices.

Class Diagram

The system is structured around four interacting Java classes. Their UML-style representation is shown below, including all attributes and methods:



Appointment
- doctor : Doctor - patient : Patient - timeSlot : String - status : String
+ getDoctor() : Doctor + getPatient() : Patient + getTimeSlot() : String + getStatus() : String + setStatus(status : String) : void + displayAppointment() : void

AppointmentManager
- doctors[] : Doctor - patients[] : Patient - appointments[] : Appointment - appointmentCount : int
+ isSlotAvailable(doctorId, timeSlot) : boolean + bookAppointment(doctorId, patientId, timeSlot) : void + cancelAppointment(patientId, timeSlot) : void + displayAllAppointments() : void

Implementation

The system is implemented using four Java classes. The logic of each class is explained below (code is not pasted here — refer to source file):

1. Doctor Class

The Doctor classrepresents a hospital doctor. It stores three private attributes— doctorId, name, and specialization. These are accessed externally through public getter methods, demonstrating encapsulation. The displayInfo()

method prints the doctor's complete details to the console in aformatted manner.

2. Patient Class

The Patient class represents a hospital patient. It stores three private attributes — patientId, name, and age. Like the Doctor class, it uses private fields with public getters to enforce data hiding. The displayInfo() method outputs the patient's details to the console.

3. Appointment Class

The Appointment classis the central link in the system and demonstrates object composition. It holds a reference to a Doctor object and a Patient object, along with a timeSlot string and a status field (either "Booked" or "Cancelled"). The setStatus() method allows the status to be updated when an appointment is cancelled. The displayAppointment() method prints a formatted appointment summary.

4. AppointmentManager Class

This is the main controllerclassthat orchestrates all scheduling operations. It maintainsthree arrays — one each for doctors, patients, andappointments — along with a counter to track how manyappointments have been created. The isSlotAvailable() methodloopsthrough the appointments array to check whether a givendoctoralready has a booking at the requested time. ThebookAppointment() method first calls isSlotAvailable(), andiftheslotis free, creates a new Appointment object and addsittothe array. If the slot is taken, it prints an error. ThecancelAppointment() method searches for a matching appointmentby patient ID and time

slot, then updates its status. The displayAllAppointments() method prints a complete formatted list of all appointments.

Sample Input / Output

The following terminal output demonstrates the system performing a booking, a conflict detection, and a cancellation in sequence:

```
=== Hospital Appointment Booking System ===

>> Checking slot: Dr. Sharma | 10:00 AM
   Available slot: 10:00 AM
   Appointment booked successfully!

--- Appointment Details ---
   Doctor : Dr. Sharma (Cardiology)
   Patient : Rahul (Age: 28)
   Time    : 10:00 AM
   Status : Booked

>> Attempting same slot: Dr. Sharma | 10:00 AM
   Error: Slot already booked!

>> Cancelling: Rahul | 10:00 AM
   Appointment cancelled successfully!
   Status : Cancelled
=== All Appointments ===

   [1] Dr. Sharma | Rahul | 10:00 AM | Cancelled
   [2] Dr. Mehta | Priya | 11:30 AM | Booked
```

Future Enhancements

The current system provides a solid console-based foundation. The following enhancements are proposed for future versions to make the system more robust and production-ready:

- Database Integration: Replace in-memory arrays with a relational database such as MySQL or PostgreSQL for persistent storage across sessions. JDBC can be used to connect Java with the database.
- Graphical User Interface (GUI): Develop a user-friendly GUI using Java Swing or JavaFX, replacing the console interface with buttons, forms, and visual calendars.
- Web Application: Convert the system into a full-stack web application using Java Spring Boot for the backend and HTML/CSS/JavaScript for the frontend.
- User Authentication: Add a secure login system with role-based access — separate portals for patients, doctors, and hospital administrators.
- Appointment Reminders: Integrate email (using JavaMail API) or SMS notifications to automatically remind patients of upcoming appointments.
- Search and Filter: Allow filtering and searching of appointments by doctor name, specialization, date, or patient ID.
- Dynamic Data Structures: Replace fixed-size arrays with ArrayList or LinkedList to support unlimited appointments without predefined limits.
- Report Generation: Add functionality to export appointment summaries as PDF reports using libraries such as iText or Apache PDFBox.

- Multi-Doctor Scheduling View: Display a calendar view of all doctor schedules simultaneously for better hospital-wide resource management.

Gantt Chart

The following Gantt chart represents the project timeline, showing the planned start and end of each task across a 4-week development period:

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis & Design	■			
Class Design & UML Diagram	■	■		
Coding — Doctor & Patient Class		■		
Coding — Appointment Class		■	■	
AppointmentManager Logic			■	
Testing & Debugging			■	■
Report Writing			■	■
Final Review & Submission				■

■ = Active / In Progress

Conclusion

The Hospital Appointment Booking System successfully demonstrates the practical application of Object-Oriented Programming concepts in Java. By modeling real-world hospital entities — doctors, patients, and appointments — as interacting objects, the system provides a clean, modular, and well-structured architecture that is easy to understand, maintain, and extend.

The project highlights core OOP principles in action: encapsulation is achieved through private fields and public getter methods in the Doctor and Patient classes; object composition is demonstrated in the Appointment class which holds references to both Doctor and Patient objects; and class interaction is shown through the AppointmentManager, which coordinates all operations by using objects of the other three classes.

The scheduling logic ensures conflict-free bookings by checking slot availability before any appointment is confirmed. The cancellation feature allows appointments to be marked as cancelled, reflecting real hospital workflows. Data is efficiently managed using arrays of objects, a fundamental skill in Java programming.

Through this project, students gain hands-on experience in designing multi-class Java applications, implementing real-world business logic, managing arrays of objects, and producing structured, readable code. The system serves as a strong foundation for future enhancements including database integration, GUI development, and web-based deployment.

Overall, this project reinforces the understanding that Object-Oriented Programming is not just a programming technique but a powerful approach for modeling and solving real-world problems in a structured and scalable way.

References

[1] H. Schildt, Java: The Complete Reference, 11th ed. New York: McGraw-Hill, 2018.

[2] B. Eckel, Thinking in Java, 4th ed. Upper Saddle River: Prentice Hall, 2006.

[3] Oracle, "Java Documentation," Oracle. [Online].

Available: https://docs.oracle.com

[4] GeeksforGeeks, "Object Oriented Programming in Java,"

GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/object-oriented-programming-oops-concept-in-java/

[5] TutorialsPoint, "Java OOPs Concepts," TutorialsPoint.

[Online]. Available: https://www.tutorialspoint.com/java/java_oops_concepts.htm

[6] S. Kumar, "Design Patterns in Java Applications," in Proc. IEEE Conf. on Software Engineering, 2022, pp. 88-93.

[7] A. Sharma and R. Gupta, "Object-Oriented Approaches in Healthcare Systems," International Journal of Computer Science, vol. 9, no. 3, pp. 112-118, 2021.